Spring Boot Starter

Study Notes

Day 1 & Day 2 — Complete Reference

Auto-Configuration · @ConfigurationProperties · @Conditional Annotations · JWT Security Design

Spring Boot 3.x · Java 17+ · springdoc-openapi 2.x

This document captures everything learned during a 2-day deep-dive into building a custom Spring Boot Starter from scratch — covering JAR files, Maven Central, classpath mechanics, auto-configuration, @Conditional annotations, @ConfigurationProperties, and security filter chain design.

Contents

10. matchIfMissing behaviour
11. Combining all three @Conditional annotations
12. Full flow recap

Day 2 — Morning

13. The problem with hardcoded values
14. @ConfigurationProperties — what and why
15. Prefix binding and relaxed binding
16. Nested inner classes in properties
17. @Validated and @NotBlank — fail-fast behaviour
18. IntelliJ autocomplete with configuration-processor
19. AppConfiguration to output — full flow

Day 2 — Afternoon

20. Making SecurityFilterChain flexible
21. Configurable public endpoints via properties
22. Three levels of starter flexibility
23. Where the boundary sits — starter vs consuming project

Day 1 — How Spring Boot Auto-Configuration Works

Morning — JAR Files · Maven Central · Classpath · Auto-Configuration

1. What is a JAR File?

A JAR (Java Archive) is simply a ZIP file with a **.jar** extension. It bundles compiled **.class** files together so they can be distributed and reused. You can rename any .jar to .zip and extract it — you will find normal folders and .class files inside.

YourCode.java	Source file — you write this
YourCode.class	Compiled bytecode — Java runs this
myproject.jar	Bundled .class files — you share this

2. What is Maven Central?

Maven Central (**repo1.maven.org/maven2**) is a public file server — like the Google Play Store but for Java libraries. Developers and companies upload their JAR files there. Maven downloads them automatically when you add a dependency to pom.xml.

The URL structure is deterministic — built from groupId + artifactId + version:

```
groupId org.hibernate.orm → org/hibernate/orm artifactId hibernate-core → hibernate-core
version 6.6.45.Final → 6.6.45.Final Final URL:
https://repo1.maven.org/maven2/org/hibernate/orm/
hibernate-core/6.6.45.Final/hibernate-core-6.6.45.Final.jar
```

■ **Pro tip:** search.maven.org is the search UI. repo1.maven.org/maven2 is the actual file server. Same data, two different interfaces.

3. What is the Classpath?

The classpath is the list of locations Java looks in when your application runs — to find classes and resources. JARs stay in your **~/.m2/repository** folder; they are never copied into your project. Maven tells the JVM where to find them at runtime.

target/classes/	Your own compiled code
~/.m2/repository/...jar	Every downloaded dependency
Both together	= the full classpath your app sees

4. META-INF and AutoConfiguration.imports

META-INF (Meta Information) is a standard Java folder inside every JAR. It holds metadata about the JAR — not application code. It is completely unrelated to Spring MVC views (which live in *src/main/resources/templates/*).

```
META-INF/ ■■■ MANIFEST.MF ← basic jar info ■■■ spring/ ■ ■■■ org.springframework.boot ←
Spring Boot reads this ■ .autoconfigure file to find all ■ .AutoConfiguration.imports
auto-config classes ■■■ spring-configuration-metadata.json ← IntelliJ autocomplete
```

■ **Pro tip:** You will NOT see this file in your own project. It lives inside downloaded JARs in .m2. You only create it yourself when building your own starter.

5. How Spring Boot Loads Auto-Configurations

@SpringBootApplication is three annotations combined. The key one is **@EnableAutoConfiguration** — it triggers the entire auto-configuration mechanism.

```
@SpringBootConfiguration @EnableAutoConfiguration // triggers auto-config scanning
@ComponentScan public @interface SpringBootApplication {}
```

The complete startup flow:

App starts → @SpringBootApplication fires
↓
@EnableAutoConfiguration activates
↓
Spring scans ALL jars on classpath for AutoConfiguration.imports
↓
Finds list of auto-configuration class names
↓
For each class — checks @Conditional conditions
↓
Conditions pass → beans created in application context
↓
Conditions fail → class skipped entirely
↓
App fully started

6. @ConditionalOnClass — Is the Library Present?

Checks whether a specific class exists on the classpath. If the JAR containing that class was not added as a dependency, the class is absent, the condition fails, and the entire auto-configuration is silently skipped.

```
@AutoConfiguration @ConditionalOnClass(DataSource.class) // only if DB driver jar is present
public class DataSourceAutoConfiguration { @Bean public DataSource dataSource() { ... } }
```

Real example: **mysql-connector-j.jar** has NO AutoConfiguration.imports file. It just puts *com.mysql.cj.jdbc.Driver* on the classpath. Spring Boot's DataSourceAutoConfiguration detects it via @ConditionalOnClass and automatically configures the DataSource bean.

7. @ConditionalOnMissingBean — Has the Developer Overridden This?

This checks whether a **bean** (a live managed object) already exists in the application context — **not** whether a class exists on the classpath. If the consuming project has already defined their own bean of that type, your starter steps aside and uses theirs.

```
@Bean @ConditionalOnMissingBean // step aside if developer defined their own public JwtFilter jwtFilter() { return new JwtFilter(); }
```

@ConditionalOnClass	Checks the classpath — is a .class file present in a JAR?
@ConditionalOnMissingBean	Checks the app context — has a bean of this type been created?

8. Class vs Bean — The Critical Difference

Class	Compiled blueprint sitting in a JAR. Not running. Just available on the classpath. Example: JwtFilter.class inside your starter JAR.
Bean	An actual object Spring created, configured, and is managing in the application context. Spring controls its lifecycle, injects it wherever needed, shares one instance across the app.
new JwtFilter()	Just an object — you manage it, Spring knows nothing about it.
@Bean JwtFilter	A Spring-managed bean — injected automatically, one shared instance, lifecycle managed.

■ **Pro tip:** A class can exist on the classpath without ever becoming a bean. @ConditionalOnClass checks the classpath. @ConditionalOnMissingBean checks the context. Never confuse them.

Afternoon — @Conditional Annotations Deep Dive

9. @ConditionalOnProperty — Toggle Features via Properties

Activates a bean based on a value in application.properties. This is how consuming projects enable or disable features in your starter without touching any Java code.

```
@Bean @ConditionalOnProperty( name = "myapp.jwt.enabled", havingValue = "true", matchIfMissing = true // ON by default if property not set ) public JwtFilter jwtFilter() { ... }
```

10. matchIfMissing — Controlling Default Behaviour

matchIfMissing = true	Feature is ON by default. Developer must explicitly set the property to false to disable it. Use this for security features — secure by default.
matchIfMissing = false	Feature is OFF by default. Developer must explicitly set the property to true to enable it. Use this for optional features that not every project needs.

11. Combining All Three @Conditional Annotations

Production starters combine multiple conditions. All conditions must pass for the bean to be created. If any one fails, the bean is skipped.

```
@Configuration @ConditionalOnClass(name = "io.jsonwebtoken.Jwts") // JWT lib on classpath? @ConditionalOnProperty( name = "myapp.jwt.enabled", havingValue = "true", matchIfMissing = true ) public class JwtAutoConfiguration { @Bean @ConditionalOnMissingBean(name = "jwtFilter") // developer override? public JwtFilter jwtFilter(AppProperties props) { return new JwtFilter(props.getJwt().getSecret()); } }
```

@ConditionalOnClass	Guard 1 — Is the JWT library even available?
@ConditionalOnProperty	Guard 2 — Has the developer enabled JWT in properties?
@ConditionalOnMissingBean	Guard 3 — Has the developer defined their own filter?

12. The Complete Auto-Configuration Flow for Your Starter

Developer adds your starter JAR to pom.xml
↓
Maven downloads JAR from Maven Central to ~/.m2
↓
JAR added to application classpath
↓
App starts → Spring finds AutoConfiguration.imports inside your JAR
↓
Finds JwtAutoConfiguration listed there

↓
@ConditionalOnClass — is io.jsonwebtoken.Jwts on classpath? YES
↓
@ConditionalOnProperty — is myapp.jwt.enabled=true? YES (matchIfMissing=true)
↓
@ConditionalOnMissingBean — has developer defined jwtFilter? NO
↓
All three conditions pass → JwtFilter bean created and managed
↓
Consuming project injects JwtFilter wherever needed — zero config

■ **Pro tip:** Your starter does nothing if conditions fail. It is always non-invasive — it steps aside when the developer takes over.

Day 2 — Configuration & Security Flexibility

Morning — @ConfigurationProperties Deep Dive

13. The Problem With Hardcoded Values

If your starter hardcodes JWT secrets and expiry values, every consuming project is stuck with them. You need a clean mechanism for consuming projects to pass their own values in without touching starter code.

Before

```
// Hardcoded — useless in real world public class JwtUtil { private String secret = "mySecretKey"; private long expiry = 86400000; }
```

After

```
// Configurable — each project sets // their own values in // application.properties myapp.jwt.secret=prodSecret myapp.jwt.expiry=3600000
```

14. @ConfigurationProperties — What and Why

Binds a group of related properties from application.properties to a single clean Java class. Far superior to @Value for starters because it supports: grouping, defaults, validation, nested classes, and IntelliJ autocomplete.

```
@ConfigurationProperties(prefix = "myapp.jwt") public class JwtProperties { private String secret = "defaultSecret"; // default value private long expiry = 86400000L; // default 24h private String header = "Authorization"; // getters and setters — Spring uses these via reflection }
```

@Value	One property per field. No grouping, no defaults without ugly syntax, no validation.
@ConfigurationProperties	All related properties in one class. Defaults, validation, nesting, autocomplete — everything.

15. Prefix Binding and Relaxed Binding

Spring Boot uses **relaxed binding** — it automatically converts between naming conventions. You write kebab-case in application.properties, and Spring maps it to camelCase Java fields via setters.

myapp.app-name	→ binds to → private String appName
myapp.max-users	→ binds to → private Integer maxUsers
myapp.jwt.expiry	→ binds to → Jwt inner class → private Long expiry

■ **Note:** Always use lowercase kebab-case for your prefix. myApp is invalid — Spring Boot will refuse to start with: 'Invalid characters: A. Canonical names should be kebab-case'.

16. Nested Inner Classes in Properties

Group related properties using static inner classes. Three rules: always static, always declare as a field in the parent, always add getters and setters.

```
@ConfigurationProperties(prefix = "myapp") public class AppProperties { private Jwt jwt = new Jwt(); // declared as field private Audit audit = new Audit(); // declared as field public Jwt getJwt() { return jwt; } public void setJwt(Jwt jwt) { this.jwt = jwt; } public Audit getAudit() { return audit; } public void setAudit(Audit a) { this.audit = a; } public static class Jwt { // MUST be static private String secret; private Long expiry = 86400000L; private boolean enabled = true; // getters and setters } public static class Audit { // MUST be static private boolean enabled = true; private String tableName = "audit_log"; // getters and setters } }
```

Why must inner classes be static?

Non-static inner classes in Java require an instance of the outer class to be created first. Spring's reflection-based binding tries to instantiate inner classes directly — it cannot do this with non-static inner classes. Making them static cuts the link to the outer class, allowing Spring to instantiate them independently.

Non-static inner class	new Jwt() is ILLEGAL without outer AppProperties instance → Spring binding fails
Static inner class	new Jwt() works perfectly standalone → Spring binds properties cleanly

17. @Validated and @NotBlank — Fail-Fast Behaviour

Without validation, a consuming project could forget to set the JWT secret and your starter would silently use a weak default in production. Add @Validated to make your starter fail at startup with a clear, actionable error message.

```
@Validated @ConfigurationProperties(prefix = "myapp.jwt") public class JwtProperties { @NotBlank(message = "JWT secret must be set — add myapp.jwt.secret to application.properties") private String secret; @Min(value = 300000, message = "JWT expiry must be at least 5 minutes") private long expiry = 86400000L; }
```

Required dependency in pom.xml:

```
<dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-validation</artifactId> </dependency>
```

18. IntelliJ Autocomplete — Configuration Processor

Add this dependency so developers using your starter get IntelliJ autocomplete when typing in application.properties. The processor generates META-INF/spring-configuration-metadata.json at compile time. The optional=true means it is not pulled into consuming projects.

```
<dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-configuration-processor</artifactId> <optional>true</optional> </dependency>
```

19. AppConfiguration to Output — The Complete Flow

App starts → Spring scans packages → finds AppConfiguration (@Configuration)

↓
Sees @EnableConfigurationProperties(AppProperties.class)
↓
Creates AppProperties instance via reflection
↓
Reads application.properties — finds all keys starting with 'myapp'
↓
Calls matching setters: setAppName(), setJwt(), setAudit() etc.
↓
AppProperties bean fully populated — registered in application context
↓
Spring sees @Bean appInfo method needs AppProperties parameter
↓
Finds AppProperties bean in context → injects it as argument
↓
Method executes → values printed → 'appInfo' String returned as bean
↓
App fully started

■ **Pro tip:** You never call new AppProperties() or appInfo() yourself. Spring does it all — reading annotations, creating instances, injecting dependencies, calling methods — in the right order.

Afternoon — Security Filter Chain Flexibility

20. Making SecurityFilterChain Flexible

A rigid SecurityFilterChain hardcoded in your starter is useless. Every consuming project has different security needs. The solution: expose it as a @ConditionalOnMissingBean so projects can override it completely.

```
@Bean @ConditionalOnMissingBean(SecurityFilterChain.class) // override-able public
SecurityFilterChain securityFilterChain( HttpSecurity http, AppProperties properties) throws
Exception { List<String> publicEndpoints = properties.getJwt().getPublicEndpoints(); http
.csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> { publicEndpoints.forEach(ep ->
auth.requestMatchers(ep).permitAll()); auth.anyRequest().authenticated(); }); return
http.build(); }
```

21. Configurable Public Endpoints via Properties

Add a publicEndpoints list to your Jwt inner class with sensible defaults. Consuming projects override them in application.properties — no Java code needed.

```
// In AppProperties.Jwt inner class: private List<String> publicEndpoints = List.of(
"/api/auth/**", "/v3/api-docs/**", "/swagger-ui/**" ); // Consuming project's
application.properties: myapp.jwt.public-endpoints=/api/auth/**,/api/public/**,/health
```

22. The Three Levels of Starter Flexibility

Every professional starter should offer consuming projects three levels of control:

The most important architectural decision for your starter. Your starter handles common plumbing. Business logic belongs in the consuming project.

Your starter owns	JWT filter logic (generate, validate, extract claims). Security filter chain default. Property binding. Auto-configuration wiring.
Consuming project owns	Role-based access rules (ADMIN, USER). Which specific endpoints are public. Custom JWT claims. Business-specific security logic.
The clean handoff	Consuming project overrides SecurityFilterChain but still INJECTS your JwtAuthFilter. They plug your JWT logic into their own security rules.

Quick Reference — All Annotations at a Glance

Annotation	Where Used	What It Does
`@AutoConfiguration`	Starter config class	Marks as auto-config. Found via AutoConfiguration.imports, NOT compon
`@Configuration`	Your own app	Standard config class. Found via component scanning.
`@EnableConfigurationProperties`	Auto-config class	Registers a @ConfigurationProperties class as a bean and activates prop
`@ConfigurationProperties`	Properties class	Binds application.properties keys with matching prefix to this class via sett
`@ConditionalOnClass`	Config/Bean method	Only activates if specified class exists on the classpath.
`@ConditionalOnMissingBean`	Bean method	Only creates this bean if no bean of that type/name exists in the app conte
`@ConditionalOnProperty`	Config/Bean method	Only activates if specified property has the specified value in application.p
`@Validated`	Properties class	Enables Bean Validation on the properties class. Fails fast at startup if inv
`@NotBlank / @Min etc.`	Properties fields	Validation constraints. App refuses to start if violated — with clear error m
`@Hidden`	Controller / Method	springdoc: hides entire controller or single endpoint from Swagger UI.

Common Mistakes to Avoid

What's Next — Day 3 Preview

On Day 3 you take everything from Day 1 and Day 2 and build the actual starter JAR. Here is what that involves:

Step 1	Create a new Maven project — plain JAR, no main class, no @SpringBootApplication.
Step 2	Move your JWT filter, JwtUtil, and AOP audit aspect into the new project.
Step 3	Create JwtAutoConfiguration and AuditAutoConfiguration with @ConditionalOnMissingBean.
Step 4	Create AppProperties with nested Jwt and Audit inner classes.
Step 5	Create META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports and list your config classes.
Step 6	Run mvn install to put your starter in ~/.m2 locally.
Step 7	Add your starter as a dependency in your ride-sharing project and test it.
Goal	Plug into a second unrelated project with minimal friction — that is when it is done.